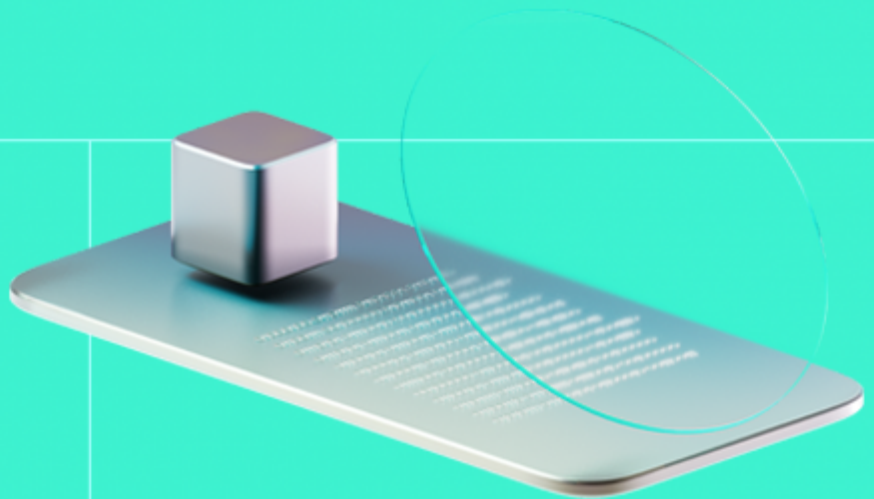




Smart Contract Code Review And Security Analysis Report

Customer: Launchly

Date: 28/01/2026



We express our gratitude to the Launchly team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

Launchly implements a Uniswap v4 hook that intercepts swap execution to collect a fixed 2% fee on swaps involving the chain's native currency, withdrawing the fee via the PoolManager and forwarding it to an immutable fee recipient through before-swap and after-swap accounting adjustments.

Document

Name	Smart Contract Code Review and Security Analysis Report for Launchly
Audited By	Panagiotis Konstantinidis
Approved By	
Website	https://launchly.app
Changelog	13/01/2026 - Preliminary Report 28/01/2026 - Final Report
Platform	Binance Smart Chain (BSC)
Language	Solidity
Tags	DEX, AMM
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Repository	https://github.com/ashutoshchaturvedi08/audit-v4-hook
Initial Commit	db6d9f0e9cd9500527aa3a6d916d0368f55b9ec1
Final Commit	a8f07f47bb03a49c45757b6a1d375ca3e2e68b61

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

5	3	2	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	1
Medium	2
Low	1

Vulnerability	Severity	Status
F-2026-14653 - Incorrect BeforeSwapDelta Mapping Causes Fee To Be Applied To Wrong Swap Side	High	Fixed
F-2026-14654 - AfterSwap Return Delta Applied To Unspecified Currency Breaks Exact-Output Fee Charging	Medium	Fixed
F-2026-14661 - Exact-Output Swaps With Native Input Revert Due To Unsupported amountSpecified Sign	Medium	Accepted
F-2026-14665 - Fee Recipient Can Cause Swap Reverts Leading To Operational Denial Of Servic	Low	Accepted
F-2026-14690 - Floating Pragma	Info	Fixed

Documentation quality

- Functional requirements and fee policy assumptions are not clearly specified.
- Technical design details are not documented.
- Function-level NatSpec comments are minimal and do not fully describe intended behavior and revert conditions.

Code quality

- The implementation leverages standard Uniswap v4 core/periphery interfaces and libraries rather than duplicating common functionality.
- The code is concise and generally follows Solidity best practices.
- Some protocol-specific assumptions are encoded implicitly in logic rather than expressed as explicit invariants.

Test coverage

Code coverage of the project is **44.45%** (branch coverage).

- Deployment, pool initialization, liquidity minting, and basic swap flows are covered.
- Fee collection is exercised for native-input swaps and verified via balance/event checks, but edge cases around exact-output and hook delta semantics are not covered.
- Multi-user or adversarial interaction scenarios are not tested.

Table of Contents

System Overview	6
Privileged Roles	6
Potential Risks	7
Findings	8
Vulnerability Details	8
Disclaimers	22
Appendix 1. Definitions	23
Severities	23
Potential Risks	23
Appendix 2. Scope	24
Appendix 3. Additional Valuables	25

System Overview

Launchly is a Uniswap v4 hook-based fee module that includes the `LaunchlyBNBHook` contract, designed to attach to a Uniswap v4 pool and impose an additional hook-level fee on swaps that involve the chain's native currency. The hook is configured to run on the swap lifecycle only (before and after swap callbacks) and forwards collected fees to an immutable `FEE_RECIPIENT` address set at deployment time.

Operationally, the hook detects whether the native currency is on the input side or the output side of a swap, computes a 2% fee based on the relevant amount, and collects that fee using `poolManager.take()` function. To keep the swap's accounting consistent with the fee extraction, it returns hook deltas (`BeforeSwapDelta` and the `afterSwap` delta) that are intended to adjust the swap's settlement to reflect the deducted fee.

The contract intentionally rejects direct native currency deposits via `receive()` and `fallback()` to prevent accidental funds from being sent to the hook address, keeping the hook's role limited to fee collection during PoolManager-mediated swap execution.

Privileged roles

- `LaunchlyBNBHook` has no owner or admin roles and no post-deployment privileged configuration functions.
- The `FEE_RECIPIENT` is set once during deployment and is the immutable address to which all hook fees are transferred during swap execution.

Potential Risks

- **Dependence on Uniswap v4 Core Contracts:** The correctness and safety of `LaunchLyBNBHook` relies heavily on Uniswap v4 core contracts (e.g., `PoolManager`, hook execution logic, delta accounting). Any vulnerabilities, specification changes, or unexpected behaviour in these external components may directly impact the hook's correctness, fee accounting, or availability.
- **Reliance on Uniswap v4 Hook Semantics:** The hook's logic depends on precise Uniswap v4 semantics, such as specified/unspecified currency handling, delta application rules, and swap lifecycle guarantees. Misinterpretation or future changes in these semantics could result in incorrect fee application or swap reverts.
- **Unaudited or Independently Audited Dependencies:** The audit does not cover the internal implementation of Uniswap v4 core or periphery libraries used by the contract. Any issues in these external libraries fall outside the scope of this assessment.
- **Fee Recipient Availability Risk:** The system relies on a single, immutable `FEE_RECIPIENT` address to receive all collected fees. If this address is misconfigured (e.g., a contract that cannot receive native currency or reverts on receipt), swaps involving fee collection may revert, leading to partial or complete loss of protocol availability.
- **No Recovery Mechanism:** Because the fee recipient is immutable and there are no administrative controls, there is no on-chain mechanism to recover from a misconfigured fee recipient without redeploying the hook and migrating liquidity.
- **Rounding-Based Fee Evasion for Small Swaps:** Fee calculation uses integer division and explicitly skips fee collection when the calculated fee rounds down to zero. This allows very small swaps to avoid paying hook fees entirely. While typically mitigated by gas costs, this may allow fee minimization strategies through trade splitting.
- **Asymmetric Fee Application Model:** Fees are charged pre-swap when native currency is the input and post-swap when native currency is the output. This asymmetric design may lead to different effective fee bases for economically similar swaps routed through different paths or swap modes, potentially affecting predictability for integrators and users.
- **Lack of Administrative Controls:** The contract intentionally has no owner, admin, pause, or upgrade functionality. While this reduces governance risk, it also means that any discovered issues, economic misconfigurations, or integration incompatibilities cannot be addressed without redeploying the hook and updating dependent pools.

Findings

Vulnerability Details

[F-2026-14653](#) - Incorrect BeforeSwapDelta Mapping Causes Fee To Be Applied To Wrong Swap Side - High

Description:

In Uniswap v4, hooks can return a `BeforeSwapDelta` to adjust swap accounting before execution.

Importantly, `BeforeSwapDelta` is defined in terms of **specified** and **unspecified** currencies, not `currency0` and `currency1`.

Key Uniswap v4 concepts required to understand this issue:

- A swap has **two modes**:
 - **Exact input** (`amountSpecified < 0`)
 - **Exact output** (`amountSpecified > 0`)
- In **exact-input swaps**:
 - **specified currency** = input token
 - **unspecified currency** = output token
- `BeforeSwapDelta(specifiedDelta, unspecifiedDelta)` applies balance adjustments based on this rule.

This contract explicitly restricts `_beforeSwap` to **exact-input swaps only**:

```
require(params.amountSpecified < 0, "unexpected amount sign");
```

Therefore, in this function the **specified** currency is always the **input token**.

In the contract, when the native currency is the input, the hook:

1. Takes a 2% fee from the native input currency using the `poolManager.take()` function.
2. Returns a `BeforeSwapDelta` intended to account for that fee.

However, the returned delta is chosen based on whether the native currency is `currency0` or `currency1`, instead of whether it is the **specified** currency.

```
if (Currency.unwrap(nativeCurrency) == Currency.unwrap(key.currency0)) {  
    return (  
        BaseHook.beforeSwap.selector,  
        toBeforeSwapDelta(int128(int256(feeAmount)), 0),  
        0  
    )  
}
```

```

    );
  } else {
    return (
      BaseHook.beforeSwap.selector,
      toBeforeSwapDelta(0, int128(int256(feeAmount))),
      0
    );
  }
}

```

This logic assumes:

- first delta field corresponds to `currency0`
- second delta field corresponds to `currency1`

In Uniswap v4, this assumption is **incorrect**. The two fields correspond to **specified** and **unspecified** currencies.

For example, let's assume:

- `currency0 = ERC20`
- `currency1 = native`
- `zeroForOne == false` (so swap `currency1 to currency0`)
- exact-input swap

Then:

- input token = `currency1` (native)
- specified currency = native
- the fee is correctly taken from native input

To deduct the fee from the user's exact input, the hook must apply the delta to the **specified** component:

```
BeforeSwapDelta(specifiedDelta = feeAmount, unspecifiedDelta = 0)
```

Instead, the current code returns:

```
BeforeSwapDelta(0, feeAmount)
```

This applies the delta to the **unspecified (output)** side, while the fee was taken from the input. The specified amount sent to the pool is therefore not reduced, and accounting becomes inconsistent.

As a result:

- The fee delta is applied to the wrong swap component.
- The specified amount sent to the pool does not correctly reflect the deducted fee.
- This creates an accounting mismatch between the hook's `take()` call and the returned delta.
- Depending on PoolManager invariant enforcement, this may:

- revert the swap (denial of service), or
- mischarge the user by affecting the wrong leg of the swap.

This issue occurs even though the hook restricts itself to exact-input swaps.

Assets:

- src/LaunchlyBNBHook.sol
[<https://github.com/ashutoshchaturvedi08/audit-v4-hook>]

Status:

Fixed

Classification

Impact Rate:	4/5
Likelihood Rate:	4/5
Exploitability:	Independent
Complexity:	Simple
Severity:	High

Recommendations

Remediation: The hook must return `BeforeSwapDelta` based on **specified/unspecified** semantics, not on `currency0` / `currency1`.

Since `_beforeSwap` only allows exact-input swaps, the fee is always taken from the **specified** currency.

Therefore, the delta should always be applied to the **specified** side, regardless of whether the token is `currency0` or `currency1`.

Example Fix. Replace the conditional delta logic with keeping only:

```
return (  
    BaseHook.beforeSwap.selector,  
    toBeforeSwapDelta(int128(int256(feeAmount)), 0),  
    0  
);
```

This correctly expresses:

- “the specified (input) currency pays the fee”
- independent of pool token ordering

If support for exact-output swaps is added in the future, the delta placement must then be adjusted based on swap mode as well.

Resolution:

Fixed in commit `a8f07f4`. The `beforeSwap` fee accounting logic was updated to correctly apply `BeforeSwapDelta` using Uniswap v4's specified/unspecified semantics rather than relying on `currency0` / `currency1` ordering. The hook now always applies the fee delta to the specified (input) currency for exact-input swaps, ensuring that the accounting delta matches the currency from which the fee is actually taken. This eliminates swap accounting mismatches and prevents potential reverts or incorrect fee application caused by applying the delta to the wrong swap leg.

Evidences

PoC: Incorrect BeforeSwapDelta Application Due To Currency-Based Mapping Instead Of Specified/Unspecified Semantics

Reproduce:

- Deploy a mock `IPoolManager` that records fee withdrawals triggered by the hook.
- Deploy `LaunchlyBNBHook` with valid hook flags using `HookMiner` to satisfy Uniswap v4 hook address requirements.
- Construct a `PoolKey` where the native currency appears as `currency1` (non-canonical ordering) to exercise the branch that depends on `currency0` / `currency1` comparison.
- Configure an exact-input swap (`amountSpecified < 0`) with `zeroForOne = false`, making the native currency the input and therefore the specified token.
- Invoke the hook's `beforeSwap` callback via the mock pool manager.
- Observe that the hook:
 - correctly withdraws the 2% fee from the native input currency, but
 - returns a `BeforeSwapDelta` where the fee is applied to the unspecified (output) component instead of the specified (input) component.

The PoC demonstrates that when native currency is the input in an exact-input swap, `LaunchlyBNBHook` applies the fee delta to the unspecified side rather than the specified side, creating an accounting mismatch between the fee withdrawn and the returned `BeforeSwapDelta`.

Results:

```
root@localhost:~/ashutoshchaturvedi08__audit-v4-hook-main# forge test --match-test test_beforeSwapDeltaUsesCurrencyOrderInsteadOfSpecified -vvvv
[..] Compiling...
No files changed, compilation skipped

Ran 1 test for test/BeforeSwapDeltaPoC.t.sol:BeforeSwapDeltaPoC
[PASS] test_beforeSwapDeltaUsesCurrencyOrderInsteadOfSpecified() (gas: 116502832)
```

Logs:

```
mode: exact-in, zeroForOne=false
fee
2000000000000000000
take
0x0000000000000000000000000000000000000000000000000000000000000000
expected: specified
2000000000000000000
expected: unspecified
0
actual: specified
0
actual: unspecified
2000000000000000000
```

Files:

BeforeSwapDeltaPoC.t.sol

[F-2026-14654](#) - AfterSwap Return Delta Applied To Unspecified Currency Breaks Exact-Output Fee Charging - Medium

Description:

In Uniswap v4, when a hook enables `afterSwapReturnDelta`, the `_afterSwap` callback must return an `int128` delta.

This returned value is always interpreted as a delta on the “unspecified” currency, i.e., the currency that is **not** tied to `SwapParams.amountSpecified`.

The “unspecified” currency depends on swap mode:

- Exact input (`amountSpecified < 0`): unspecified = output token
- Exact output (`amountSpecified > 0`): unspecified = input token

Within the contract, when native currency is the output, the hook calculates a 2% fee on the native output amount, takes that fee from the native currency via `poolManager.take()` function, and returns the fee amount as the `afterSwap` delta:

```
Currency nativeCurrency = !params.zeroForOne ? key.currency0 : key.currency1;
poolManager.take(nativeCurrency, FEE_RECIPIENT, feeAmount);
emit FeeCollected(Currency.unwrap(nativeCurrency), feeAmount, false);

return (BaseHook.afterSwap.selector, int128(int256(feeAmount)));
```

This is consistent for exact-input swaps because the unspecified currency is the output token, so the returned delta is applied to the output leg (native), matching the currency taken.

However, for exact-output swaps:

- specified = output token (native)
- unspecified = input token (non-native)

The hook still takes the fee from native output, but `BaseHook.afterSwap` will apply the returned delta to the unspecified leg (input). This causes a mismatch between the currency actually taken and the currency being adjusted by the returned delta. Exact-output swaps are only blocked in `_beforeSwap` when native is the input. Exact-output swaps where native is the output are still reachable and will execute this `_afterSwap` path.

As a result:

- The hook’s fee extraction (`take` in native output) and the returned delta (applied to input in exact-output mode) are inconsistent.
- This can create an accounting imbalance during settlement and may:
 - revert the swap (denial of service for exact-output swaps with native output), or

- mischarge by charging the fee on the wrong leg depending on settlement behaviour.

Assets:

- src/LaunchlyBNBHook.sol
[<https://github.com/ashutoshchaturvedi08/audit-v4-hook>]

Status:**Fixed**

Classification

Impact Rate:	3/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations**Remediation:**

Ensure the fee is taken from the same leg that the `afterSwap` return delta is applied to.
A minimal fix consistent with the current design is to disallow exact-output swaps in `_afterSwap`:

Example fix:

```
require(params.amountSpecified < 0, "exact-output not supported");
```

Alternatively, if exact-output support is required, charge the fee on the unspecified leg when `amountSpecified > 0` (i.e., charge input instead of output in that mode) or redesign fee collection so `take()` and the returned delta always refer to the same currency leg.

Resolution:

Fixed in commit `a8f07f4`. The hook now explicitly disallows exact-output swaps in `afterSwap` (`require(amountSpecified < 0)`), preventing the scenario where the returned `afterSwap` delta is applied to the unspecified (input) currency while the fee is taken from native output, which previously could cause accounting mismatches and swap reverts.

[F-2026-14661](#) - Exact-Output Swaps With Native Input Revert Due To Unsupported amountSpecified Sign - Medium

Description:

`LaunchlyBNBHook` applies a 2% hook fee on swaps where the native currency (represented as `Currency.unwrap() == address(0)`) is the input side. This logic is implemented in `_beforeSwap` and is executed because the hook enables `beforeSwap` and `beforeSwapReturnDelta`.

The hook assumes swaps are executed in exact-input mode and enforces this assumption only when native currency is the input side (`isNativeInput == true`) by requiring `SwapParams.amountSpecified < 0`. In Uniswap v4, the sign of `amountSpecified` determines swap mode: negative for exact-input and positive for exact-output. Therefore, any exact-output swap where native is the input will revert, even if the pool supports exact-output swaps. This creates a functional denial of service for the "native-input exact-output" swap type and makes the fee behaviour inconsistent across swap modes.

Importantly, this revert is not triggered for all exact-output swaps. Exact-output swaps where native is the output side do not enter this `require` branch and can still reach `_afterSwap`.

In practice:

1. A user (or router) submits an exact-output swap (`params.amountSpecified > 0`).
2. The swap is configured such that the native currency is the input side (`isNativeInput == true`).
3. `_beforeSwap` executes `require(params.amountSpecified < 0, "unexpected amount sign");` and reverts.
4. The swap fails and cannot be executed in exact-output mode for native-input paths.

```
if (!isNativeInput) {
    return (BaseHook.beforeSwap.selector, toBeforeSwapDelta(0, 0), 0);
}

// Defensive check: amountSpecified must be negative for exact-input path
require(params.amountSpecified < 0, "unexpected amount sign");
```

As a result:

- Exact-output swaps with native input are hard-reverted (DoS for that swap type).
- Integrations that rely on exact-output swaps (e.g., "receive exactly X tokens") may fail when the route requires native input.

- Native-input swaps are supported only for exact-input mode, while native-output exact-output swaps may still proceed (and are handled separately in `_afterSwap`).

Assets:

- `src/LaunchlyBNBHook.sol`
[<https://github.com/ashutoshchaturvedi08/audit-v4-hook>]

Status:**Accepted****Classification**

Impact Rate:	3/5
Likelihood Rate:	4/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations**Remediation:**

Either explicitly document and enforce “exact-input only” behaviour for native-input swaps, or add support for exact-output swaps where native is the input. If the intent is to charge fees regardless of swap mode, the fee should be computed using realized amounts for exact-output mode (which typically cannot be derived directly from `amountSpecified` in `_beforeSwap`).

Example fix:

```
if (isNativeInput) {
    if (params.amountSpecified < 0) {
        // exact input
        inputAmount = uint256(-params.amountSpecified);
        chargeFeeOnInput(inputAmount);
    } else {
        // exact output: either explicitly disallow with a clear error
        revert("exact-output native-input not supported");

        // or support it by charging based on realized input amount
        // realizedInput = deriveRealizedInput(...);
        // chargeFeeOnInput(realizedInput);
    }
}
```

Resolution:

Accepted in commit [a8f07f4](#). The code now uses a clearer and explicit revert reason to indicate that exact-output swaps with native input are intentionally unsupported. However, the underlying limitation remains and native-input exact-output swaps are still not supported by design.

F-2026-14665 - Fee Recipient Can Cause Swap Reverts Leading To Operational Denial Of Service - Low

Description:

`LaunchlyBNBHook` collects a 2% hook fee on swaps involving the native currency. Fee collection is implemented in both `_beforeSwap` (for native-input swaps) and `_afterSwap` (for native-output swaps) by calling `poolManager.take()` to transfer the fee directly to an immutable `FEE_RECIPIENT` address specified at deployment.

The hook assumes that `FEE_RECIPIENT` can always successfully receive the native currency. However, if `FEE_RECIPIENT` is a contract that reverts on receiving native currency (e.g., missing or reverting `receive` / `fallback`, or containing restrictive logic), the `poolManager.take()` call will revert. Because this call occurs during swap execution, such a revert will cause the entire swap to fail. This creates an operational denial-of-service risk tied to the fee recipient's implementation or configuration rather than to user behaviour.

Specifically:

1. The protocol deploys the hook with `FEE_RECIPIENT` set to a contract address.
2. That contract either does not accept native currency or later changes behaviour (e.g., via upgrade) to revert on receipt.
3. A user executes a swap that triggers fee collection (native input or native output).
4. `_beforeSwap` or `_afterSwap` calls `poolManager.take(nativeCurrency, FEE_RECIPIENT, feeAmount)`.
5. The transfer reverts, causing the entire swap to revert.
6. All swaps requiring fee collection become unexecutable.

```
poolManager.take(nativeCurrency, FEE_RECIPIENT, feeAmount);
```

As a result:

- Swaps involving the native currency can be permanently blocked.
- The protocol's availability depends on the correct and continued behaviour of an external address.
- This risk persists even though `FEE_RECIPIENT` is immutable, as immutability does not guarantee safe receiving logic.

Assets:

- `src/LaunchlyBNBHook.sol`
[<https://github.com/ashutoshchaturvedi08/audit-v4-hook>]

Status:

Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Ensure that `FEE_RECIPIENT` is guaranteed to accept native currency under all conditions. This can be achieved by:

- Restricting `FEE_RECIPIENT` to an externally owned account (EOA), or
- Using a minimal, non-upgradeable receiver contract with a permissive `receive()` function, or
- Adding deployment-time validation to ensure the recipient can accept native transfers.

If protocol governance or upgradeability is expected, consider designing a controlled mechanism to rotate the fee recipient to recover from misconfiguration, while carefully managing trust and access control.

Example fix:

```
constructor(IPoolManager _poolManager, address _feeRecipient) BaseHook(_poolManager) {
    require(_feeRecipient != address(0), "fee recipient 0");
    // Optional: enforce EOA or known-safe receiver
    // require(_feeRecipient.code.length == 0, "recipient must be EOA");
    FEE_RECIPIENT = _feeRecipient;
}
```

Resolution:

Accepted. The client is aware of the finding and stated:

We acknowledge this operational risk and have ensured that the fee recipient is a known, safe address capable of receiving native currency without reverting. The recipient is immutable and controlled by the protocol, preventing misconfiguration. If future flexibility is required, we will introduce a governed rotation mechanism with proper safeguards.

F-2026-14690 - Floating Pragma - Info

Description:

In Solidity development, the pragma directive specifies the compiler version to be used, ensuring consistent compilation and reducing the risk of issues caused by version changes. However, using a floating pragma (e.g., `^0.8.x`) introduces uncertainty, as it allows the contract to be compiled with any version within the specified range. This can result in discrepancies between the compiler used in testing and the one used during deployment, increasing the likelihood of vulnerabilities or unexpected behavior due to changes in compiler versions.

The project currently uses floating pragma declarations (`^0.8.26`) in its Solidity contracts. This increases the risk of deploying with a compiler version different from the one tested, potentially reintroducing known bugs from older versions or causing unexpected behavior with newer versions. These inconsistencies could result in security vulnerabilities, system instability, or financial loss. Locking the pragma version to a specific, tested version is essential to prevent these risks and ensure consistent contract behavior.

```
pragma solidity ^0.8.26;
```

Assets:

- `src/LaunchlyBNBHook.sol`
[<https://github.com/ashutoshchaturvedi08/audit-v4-hook>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

It is recommended to lock the pragma version to the specific version that was used during development and testing. This ensures that the contract will always be compiled with a known, stable compiler version, preventing

unexpected changes in behavior due to compiler updates. For example, instead of using ^0.8.x, explicitly define the version with a locked, stable pragma directive.

Before selecting a version, review known bugs and vulnerabilities associated with each Solidity compiler release. This can be done by referencing the official Solidity compiler release notes: [Solidity GitHub releases](#) or [Solidity Bugs by Version](#). Choose a compiler version with a good track record for stability and security.

Resolution:

Fixed in commit `a8f07f4`. The pragma version was locked to `0.8.26` to ensure consistent compilation and avoid version related discrepancies.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/ashutoshchaturvedi08/audit-v4-hook
Initial Commit	db6d9f0e9cd9500527aa3a6d916d0368f55b9ec1
Final Commit	a8f07f47bb03a49c45757b6a1d375ca3e2e68b61
Whitepaper	https://launchly.app/dashboard/docs
Requirements	README.md
Technical Requirements	README.md

Asset	Type
src/LaunchlyBNBHook.sol [https://github.com/ashutoshchaturvedi08/audit-v4-hook]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.